

論文紹介 2026/06/18

Attention is all you need

所属未入力

発表者未入力

■ アジェンダ

- 研究背景と課題
- 提案手法：Transformer
- Attentionメカニズムの詳細
- Positional Encoding と比較
- 実験設定
- 実験結果
- まとめ

— 研究背景と課題

従来のシーケンス処理モデルはRNNやCNNが主流でした。

- **既存モデルの課題**

- RNN: 計算が逐次的で並列化が困難、長いシーケンスでボトルネック
- CNN: 長距離の依存関係を捉えるための演算数が多い (ConvS2Sでは線形、ByteNetでは対数)

- **Attentionメカニズムの台頭**

- 入力/出力シーケンスの距離に関わらず依存関係をモデル化
- しかし、ほとんどのAttentionはRNNと組み合わせて使用

— 解決したい課題

Attentionメカニズムを単独で活用し、RNNやCNNの課題を克服します。

- 目標

- RNNや畳み込みを完全に排除
- Attentionメカニズムのみでシーケンス変換モデルを構築
- 並列化を大幅に向上させる
- 高品質な翻訳と高速な学習を実現

— 提案手法：Transformerの全体像

TransformerはEncoder-Decoder構造を採用しています。

- **Attentionのみのアーキテクチャ**
 - 再帰層や畳み込み層を完全に排除
 - Self-AttentionとPoint-wiseな全結合層のみで構成
- **Encoder-Decoderスタック**
 - Encoder: 入力シーケンスを連続表現にマッピング
 - Decoder: Encoderの出力を基に、出力を逐次的に生成
- **並列化と性能向上**
 - 大幅な並列化により学習時間を短縮
 - 機械翻訳タスクで高精度を達成

Output
Probabilities



Softmax

— Encoder-Decoderスタック

EncoderとDecoderはそれぞれN=6個の同一レイヤーで構成されます。

- **Encoderレイヤー**

- Multi-Head Self-Attentionメカニズム
- Position-wise Feed-Forward Network
- 各サブレイヤー後に残差接続とLayer Normalizationを適用
- 出力次元 $d_{\text{model}} = 512$

- **Decoderレイヤー**

- Masked Multi-Head Self-Attention (未来の情報を参照しない)
- Multi-Head Attention (Encoder出力へのAttention)
- Position-wise Feed-Forward Network
- 同様に残差接続とLayer Normalizationを適用

— Scaled Dot-Product Attention

Attention関数はクエリ (Q)、キー (K)、値 (V) を出力にマッピングします。

- **基本計算**

- QとKの内積を計算し、類似度を測る
- この値をスケーリング因子 $\sqrt{d_k}$ で割る
- Softmax関数を適用してAttention重みを生成
- 重みをVと乗算し、加重和を出力とする

- **数式**

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **スケーリングの理由**

- d_k が大きい場合、内積が大きくなりSoftmaxの勾配が小さくなるのを防ぐ



— Multi-Head Attention

複数のAttentionメカニズムを並列に実行し、結合します。

- **複数のAttentionヘッド**

- Q, K, Vを h 個の異なる線形投影によって変換
- それぞれの投影に対してAttention関数を並列に実行
- 各ヘッドの出力を結合し、再び線形投影して最終出力を得る

- **目的**

- 異なる表現部分空間からの情報を共同で考慮
- 単一のAttentionヘッドによる平均化の抑制

- **本研究の設定**

- $h = 8$ 個の並列Attention層（ヘッド）を使用
- 各ヘッドの次元 $d_k = d_v = d_{\text{model}}/h = 64$
- 計算コストは単一ヘッドAttentionと同等

- **数式**

— TransformerにおけるAttentionの利用

Multi-Head Attentionはモデル内で3つの方法で使用されます。

- **Encoder-Decoder Attention**

- DecoderのクエリがEncoderの出力（キーと値）を参照
- Decoderの各位置が入力シーケンスの全ての位置にAttentionを適用

- **Encoder Self-Attention**

- Encoderのクエリ、キー、値が全てEncoderの前の層の出力から供給
- Encoderの各位置がEncoderの前の層の全ての位置にAttentionを適用

- **Decoder Self-Attention**

- Decoderのクエリ、キー、値が全てDecoderの前の層の出力から供給
- 自己回帰性を保つため、未来の位置へのAttentionをマスク

— Position-wise Feed-Forward Networks (FFN)

Attention層に加えて、各層にFFNが含まれます。

- **構造**

- 全ての層で各位置に個別に同一のFFNを適用
- ReLU活性化関数を挟む2つの線形変換で構成

- **数式**

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- **次元**

- 入力・出力次元 $d_{\text{model}} = 512$
- 中間層の次元 $d_{\text{ff}} = 2048$

- **特徴**

- カーネルサイズ1の2つの畳み込みと解釈できる

— Positional Encoding

モデルにシーケンスの順序情報を注入します。

- 必要性

- Transformerは再帰や畳み込みがないため、位置情報が失われる
- 入力埋め込みと位置エンコーディングを加算

- 手法

- 異なる周波数のサイン・コサイン関数を使用
- 埋め込み次元 d_{model} と同次元

- 数式

$$\begin{aligned} P E_{(\text{pos}, 2i)} &= \sin(\text{pos}/10000^{2i/d_{\text{model}}}) \\ P E_{(\text{pos}, 2i+1)} &= \cos(\text{pos}/10000^{2i/d_{\text{model}}}) \end{aligned}$$

- pos: 位置、i: 次元

- 利点

- モデルが相対位置を容易に学習できると仮定

— Self-Attentionの利点

RNNsやCNNsと比較して、Self-Attentionの優位性を検証します。

- **1. 計算コスト ($O(n)$ シーケンス長)** - ****Self-Attention****: $O(n^2 \cdot d)$ - $n < d$ の場合、RNNより高速 - ****Recurrent****: $O(n \cdot d^2)$ - ****Convolutional****: $O(k \cdot n \cdot d^2)$**
- **2. 並列化 (最小逐次演算)**** - ****Self-Attention****: $O(1)$ - 全ポジションが定数回の演算で接続 - ****Recurrent****: $O(n)$ - ****Convolutional****: $O(1)$
- **3. 長距離依存関係のパス長**** - ****Self-Attention****: $O(1)$ - 入力・出力間のパス長が最も短い -> 長距離依存関係の学習が容易 - ****Recurrent****: $O(n)$ - ****Convolutional****: $O(\log_k(n))$

— 実験設定

WMT 2014機械翻訳タスクと英語構文解析タスクで評価しました。

• データセット

- WMT 2014英独翻訳: 450万文、BPE、3.7万語彙
- WMT 2014英仏翻訳: 3600万文、BPE、3.2万語彙
- 英語構文解析: Penn Treebank (WSJ) 約4万文、半教師あり学習では1700万文

• モデル構成

- Baseモデル: $N = 6$, $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$, $h = 8$
- Bigモデル: $N = 6$, $d_{\text{model}} = 1024$, $d_{\text{ff}} = 4096$, $h = 16$

• 学習

- 8基のNVIDIA P100 GPUを使用
- Baseモデル: 12時間、Bigモデル: 3.5日
- Adamオプティマイザ、カスタム学習率スケジュール
- Residual Dropout、Label Smoothingを適用

— 機械翻訳タスクの結果 (WMT 2014)

Transformerは、既存の最高性能モデルを上回るBLEUスコアを達成しました。

- **英独翻訳 (WMT 2014 En-De)**
 - **Transformer (big): 28.4 BLEU**
 - 既存のアンサンブルモデルをも2.0 BLEU以上上回るSOTAを達成
- **英仏翻訳 (WMT 2014 En-Fr)**
 - **Transformer (big): 41.8 BLEU**
 - 既存の単一モデルのSOTAを確立
 - 学習コストは既存SOTAの1/4以下

[図: Table 2 (BLEU & Training Cost) を簡略化してここに挿入]

****補足**** 高い性能にもかかわらず、学習コスト（FLOPs）は既存モデルより大幅に低い。

— モデルバリエーションの評価

Transformerの各コンポーネントの重要性を検証しました。

- **Attentionヘッド数と次元**

- 単一ヘッドは性能を低下させる (0.9 BLEU減)
- ヘッドが多すぎても品質は低下
- キーの次元 d_k の削減は品質を損なう

- **モデルのサイズ**

- 大きなモデルはより良い性能を示す (BLEU向上)

- **Dropout**

- 過学習の回避に非常に有効

- **Positional Encoding**

- 提案のサイン・コサイン関数と学習された埋め込みはほぼ同等の結果
- 外挿性のためサイン・コサイン関数を採用

— 英語構文解析の結果

Transformerは構文解析タスクにも高い汎化性能を示しました。

- **WSJのみのトレーニング**

- **Transformer (4層): 91.3 F1**

- 既存のRNNベースモデルやBerkeley Parserを上回る
 - RNNシーケンス-to-シーケンスモデルが苦手とする領域で高性能

- **半教師あり学習**

- **Transformer (4層): 92.7 F1**

- 最先端のRNN文法モデルにはわずかに及ばないものの、非常に高い性能

- **タスク特性への適応**

- 出力の強い構造的制約や入力より長い出力シーケンスにも対応

— まとめ

本研究では、Attentionのみに基づくTransformerモデルを提案しました。

- **Transformerの独自性**

- 再帰層や畳み込み層を排除した初のAttentionベースシーケンス変換モデル
- Multi-Head Self-Attentionを中核として採用

- **高い性能と効率性**

- WMT 2014機械翻訳タスクで新たなSOTAを達成、既存アンサンブルをも凌駕
- 学習速度が従来のRNN/CNNベースモデルより大幅に高速

- **汎化能力**

- 英語構文解析タスクにおいても高い性能を発揮